

COMP2200/COMP6200 Practical Exercise (Week 10): Vectorisers, Limits, and Logistic Stories

School of Computing

Preparation

Choose your environment: either run locally with `uv` or use Google Colab.

1. Local with `uv`:

(a) Install `uv`.

MacOS/Linux: `curl -LsSf https://astral.sh/uv/install.sh | sh`

Windows: `powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"`

(b) Create a folder for this practical (for example, `week10-prac`) and open a terminal in it.

(c) Set up your environment:

```
uv init
```

```
uv add pandas scikit-learn matplotlib seaborn umap-learn
```

(d) Launch Jupyter with `uv run --with jupyter jupyter lab` (or ... `notebook`).

2. Google Colab:

(a) Open Colab and create a new notebook.

(b) Upload `enron_practical_sample.csv` from iLearn or this week's zip file.

(c) Install packages with `!pip install pandas scikit-learn matplotlib seaborn umap-learn`.

Load the CSV with:

```
import pandas as pd
emails = pd.read_csv("enron_practical_sample.csv")
```

Because we are going to run many different models and choose between them, we need to keep some hold-out data for reporting our final accuracy.

Double-check that you did include **umap-learn** in the instructions above. You'll need it in Part C.

Part A — Train/test split (10 min)

We will train on only 10% of the data today. That keeps the practical fast enough to experiment with, while still giving us a balanced training set.

```
from sklearn.model_selection import train_test_split
```

```
train_df, test_df = train_test_split(
    emails,
    train_size=0.10,
    stratify=emails["label"],
    random_state=2025
)
```

```
train_text = train_df["text"]
```

```

train_labels = train_df["label"]
test_text = test_df["text"]
test_labels = test_df["label"]

print(train_labels.value_counts())
print(test_labels.value_counts())

```

Use the training data for the exploratory work. The only required use of the test data is the final held-out evaluation in Part D.

Part B — Tokenisation tuning lab (15 min)

Starting with this:

```

from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
vectoriser = TfidfVectorizer(stop_words="english", ngram_range=(1, 1))
X_train = vectoriser.fit_transform(train_text)

```

1. Record the vocabulary size and the top 10 highest-IDF terms. Here's how to extract and display them:

```

# Vocabulary size
vocab_size = len(vectoriser.vocabulary_)
print(f"Vocabulary size: {vocab_size}")

# Top 10 weighted terms (highest IDF scores)
idf_series = pd.Series(vectoriser.idf_,
                       index=vectoriser.get_feature_names_out())
top_10_terms = idf_series.nlargest(10)
print("\nTop 10 weighted terms:")
print(top_10_terms)

```

2. Look carefully at the highest-IDF terms. High IDF means “rare in this training set”, not “useful for classification”. Expect some IDs, fragments, names, or junk tokens. Write down one rare token that seems meaningful and one that seems useless.
3. Rotate through the following tweaks, keeping notes on what changes and why:
 - **Character n-grams:** switch to analyzer="char_wb" with ngram_range=(3,5).
 - **Frequency trims:** set min_df=2 and max_df=0.8.
 - **Bigrams:** use ngram_range=(1,2) with default word analyzer.
4. Highlight one surprise: what token appeared/disappeared, and how could that affect logistic regression weights?

The aim is to give every group a talking point for the main modelling challenge.

Part C — UMAP visualisation (15 min)

Before diving into classification, let's visualise the email landscape to see if spam and ham naturally cluster.

1. Create a TF-IDF representation of the training emails with a reasonable feature limit. We use only the training data here so the held-out test set stays genuinely held out.

```

from sklearn.feature_extraction.text import TfidfVectorizer
import umap
import matplotlib.pyplot as plt
import seaborn as sns

```

```
# Vectorize with moderate feature budget
vectoriser = TfidfVectorizer(max_features=1000, stop_words="english")
X_tfidf = vectoriser.fit_transform(train_text)

# Reduce to 2D with UMAP
reducer = umap.UMAP(n_components=2, random_state=2025, n_neighbors=15)
embedding = reducer.fit_transform(X_tfidf)
```

2. Create a scatter plot coloring points by spam/ham. Use `alpha=0.5` for transparency:

```
plt.figure(figsize=(10, 8))
sns.scatterplot(x=embedding[:, 0], y=embedding[:, 1],
                hue=train_labels, palette=["#2ecc71", "#e74c3c"],
                alpha=0.5, s=30)
plt.title("UMAP Projection of Enron Emails (TF-IDF)")
plt.xlabel("UMAP 1")
plt.ylabel("UMAP 2")
plt.legend(title="Label")
plt.show()
```

3. Are spam and ham visually separable? Where do the classes overlap? What does this suggest about the difficulty of the classification task?
4. If you deliberately include test data in a visualisation, call it exploratory only. Do not use it to choose model settings.
5. **Optional:** Try different `n_neighbors` values (5, 15, 50) and compare how the structure changes. UMAP's neighborhood parameter controls local vs. global structure emphasis.
6. **Optional:** Try different vectorisation options (from Part B) and see if that makes a difference.

Part D — Feature budget challenge (30 min)

In this round you design the leanest email classifier that still performs. You may only keep **1,000 features**.

1. Start from this baseline pipeline:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate, StratifiedKFold
from sklearn.pipeline import Pipeline

pipeline = Pipeline([
    ("vectoriser", TfidfVectorizer(max_features=1000, stop_words="english")),
    ("model", LogisticRegression(max_iter=500))
])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=2025)
metrics = cross_validate(pipeline, train_text, train_labels,
                        cv=cv, scoring=["accuracy", "f1_macro"],
                        return_train_score=True)
```

2. Record the baseline `metrics["test_f1_macro"].mean()` score.
3. Try one or two changes from Part B, while respecting the 1,000-feature budget. Good options are `ngram_range=(1,2)`, `min_df=2`, or `sublinear_tf=True`.

4. **Optional:** Instead of 1000, how small can you make your model? `TfidfVectorizer` can take a `vocabulary=` argument where you can specify exactly which words or n-grams that it is allowed to use.
5. Once satisfied, fit the pipeline on the training data and evaluate on the held-out test data. Report accuracy, macro-F1, and the confusion matrix.
6. Document your best configuration in the shared spreadsheet: vectoriser settings, mean cross-validated F1, and one sentence explaining why it works.

Part E — Regularisation and coefficient storytelling (25 min)

Next we unpack what the logistic regression weights mean and how regularisation changes them.

Smaller C = stronger regularisation (smaller coefficients).

1. Record the vectoriser settings that delivered your best Part D score, then extend your pipeline to expose coefficients:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.utils import Bunch

best_params = {
    "max_features": 1000,
    "stop_words": "english",
    # add any other parameters that worked best in Part D
}

def train_with_c(C=1.0, penalty="l2") -> Bunch:
    pipeline = Pipeline([
        ("vectoriser", TfidfVectorizer(**best_params)),
        ("model", LogisticRegression(max_iter=1000, C=C, penalty=penalty,
                                     solver="liblinear" if penalty == "l1" else "lbfgs"))
    ])
    pipeline.fit(train_text, train_labels)
    return Bunch(
        pipeline=pipeline,
        coefficients=pipeline.named_steps["model"].coef_[0],
        vocab=pipeline.named_steps["vectoriser"].get_feature_names_out()
    )
```

2. Run the helper for $C \in \{0.1, 1, 10\}$ with L2 and for $C = 1$ with L1. Plot coefficient magnitudes (you could do this in Excel, or you could make a pandas Series and plot it)
3. For your winning configuration, list the top 5 positive and negative tokens. Translate them into plain language rules (“Emails mentioning *meeting* push the score toward a particular class”).

Part F (optional) — Error clinic and moderation huddle (10 min)

To close, we stress-test the model against borderline cases.

1. Generate a DataFrame of misclassifications on the test split with columns `text`, `true_label`, `predicted`, and `probability` (the predicted class probability).
2. Tag each misclassified email with the likely cause: **ambiguous tone**, **missing vocabulary**, or **label noise**. Capture ideas for new features or thresholds in a shared doc.

Part G – Quiz questions (10 min)

Spend some time on the Quiz questions in anticipation of Assignment 2.